

Final Project – Report

Question:

Why should we add the momentum to SGD? What is the main two improvements adam has comparing to the simple SGD? What was the best optimization algorithm and best learning rate you've found?

Summarize this section results in your report.

Answer:

We should add momentum to SGD because the momentum allows the optimizer to accumulate velocity in the direction of the previous gradients, making it move faster through flat or noisy regions therefore having more stability between epochs. **Adam's improvements:**

- Adjusts the learning rates for each parameter based of the mean and the variance of the gradients which makes it handle difference in gradients better, as if the data wasn't noisy.
- Corrects the bias, in the first learning iterations adam applies bias correction to counteract the initialization bias. Which handles better the first phases of the training process even if the bias is random and not initialized manually.

Adam with learning rate of 0.0005 worked best for me, reaching the best validation accuracy (it did overfit a bit but I believe it can be solved later).

Question:

What is the purpose of batch normalization (why do we use it)? Did the Batch normalization improve the network performance? Summarize this section results in your report.

Answer:

Batch normalization does two things, first it normalizes the data between the layers it's in, Meaning the data will be mapped to be between the values 0 and 1. So it will take less time for the next layer to process the data and also avoid having high values which can result in gradient explosion(when a large error gradients accumulate which result in a large update of the weights). The second thing it does is adds a bias and an variable to the data process between the layers it's in, and those are trainable, so it lowers the internal covariate shift problem as well by balancing more between the early layers and the layers closer to the end of the network, so the early layers can be more impactful and the later ones less (if there's a need).

Batch normalization improved the network performance greatly, mainly by improving the test and val accuracy and lowering the test and validation loss, minimizing by a little the overfitting problem which the network had (still has but less now).

Question:

Why should we use regularization? How does the regularization affect the train accuracy and loss? How does it affect the val and test accuracy and loss? What was the best regularization method? Summarize this section results in your report.

Answer:

We should use regularization to fix the overfitting problem, overfitting happened when the model learns the training dataset "too much" in a way that it learns also the noise and the finds that correlate only to the learning datasets, and not the real world (or validation and test datasets) so since regularization helps minimize and avoid the overfitting problem it lowers the train accuracy to match more closely to the test accuracy, meaning the loss of the test and validation won't increase too much while the loss of the training is decreasing therefore minimizing the overfitting problem.

regularization makes the variance of the network be effected by the weights of the network, if a certain weight is too high it's a indication that the model is overfitting, so it penalizes the model for higher weights. Dropout makes a neuron have a probability of being inactive (p) that way the network won't rely too much on certain neurons which will lead to other neurons to learn less (which makes a weight more high by being more dependable).

In my model I found dropout with probability of 0.5 to have the best results and the best validation accuracy.

Question:

What is your best architecture? what its best accuracy and loss on the test set? Compare your result to assignment 2. How did you manage to improve the model? Summarize this section results in your report.

Answer:

The end result of the best architecture i found was 4 layers per block with 64 filters per layer with hidden dim of [100,100] with batch normalization and dropout with 50%. the final accuracy and loss the network achieved are:

Accuracy: 76.07

Loss: 0.6966445505154505

compared to assignment 2, I've experimented with dropout and not only L2 regulation and also experimented with different hidden dimensions, also I added to this network batch normalization unlike the last assignment. Compared to last assignment the results of this network are much better, with %76.07 accuracy compared to %68.66 and 0.696 loss compared to 0.945. So overall a big improvement from the last assignment network.

Question:

What was the pretrained ResNet50 accuracy and loss? Is it overfit/underfit/well-fit the data? Has it got better accuracy than your model? Summarize this section results in your report.

Answer:

After training with the pretrained ResNet50 the accuracy and lost were:

Accuracy: 83.75

Loss: 0.6220886492896524

Which is better then my model which ended up with:

Accuracy: 76.07

Loss: 0.6966445505154505

since the loss is lower and the accuracy is higher it had better results then my model. The ResNet50 model did overfit a bit, more then my model since it's train loss which is 0.071 is way lower then the test loss. Probably could've been avoided if I ran only 5 epochs on it.

Question:

Write a short summerization of your attempts. What worked for you? which architecture and hyperparameters led to the best preformance? which future experiments you may want to do and why? please also suggest at least two ways you believe you may improve the preformance of the model.

Answer:

I ended up using a model with 64 filters per layer, 4 layers per block and hidden dims of size [100,100] since it had the best accuracy at the start, then learning rate of 0.0005 with Adam optimizer although Adam with LR of 0.001 and SGD with 0.1 had almost the same results. Also, the batch normalization increased the performance of my model by a lot but regularization increased it by a lot less. I ended up using regularization of dropout with probability of 0.5. the end result was %76.07 accuracy and 0.6966 loss. I actually tried a lot more models even some that didn't perform well at the start for example I've found that a model with 256 filters per layer and 4 layers per block and hidden dims of size [256,256] won't performed well without

regularization and batch normalization but after implementing those improvements the model performed just as good as the one I ended up using. So I discovered that some combinations work well although alone they don't do so well, want to experiment with different combinations and I believe that I will find a better combination and thus a better model. Also I didn't experiment with the max pooling, which also affects the model performance and could also improve the model. Last thing is ResNet50 made me realize that not all models are built evenly throughout. And maybe choosing one variable as the filters per layer isn't optimal, maybe building a combination of small networks (like ResNet50 which had much better performance) could work better.